

Social Media Analytics

Lab Practical Solutions — All 10 Experiments

Complete Python Code + Sample Output

Exp No.	Title
1	Study of Social Media Platforms, Analytics Tools & Techniques
2	Data Collection — Scraping & Crawling Social Media
3	Data Cleaning & Storage with Python & MongoDB
4	Exploratory Data Analysis & Visualization
5	Content-Based Social Media Analytics Model
6	Structure-Based Analytics — Community Detection
7	Real-Time Dashboard & Reporting Tool
8	Creative Content Design for Social Media Promotion
9	Competitor Activity Analysis
10	Text Analytics for Product/Service Improvement

Experiment 1

Study of Social Media Platforms, Analytics Tools & Techniques

Platform survey • Analytics tools • Engagement metrics

Aim

Study various social media platforms (Facebook, Twitter, YouTube), analytics tools (Facebook Insights, Google Analytics, Netlytic), analytics techniques and engagement metrics (page, post, member level), and business applications.

Theory

- **Social Media Platforms:** Facebook (2.9 B users), Twitter/X (350 M), YouTube (2.7 B), LinkedIn (950 M), Instagram (2 B).
- **Analytics Tools:** Google Analytics — tracks web traffic; Facebook Insights — page engagement; Netlytic — network analysis.
- **Engagement Metrics:** Reach, Impressions, Click-Through Rate (CTR), Engagement Rate, Sentiment Score.

Code — Platform Comparison Table

```
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

# Platform Data
platforms = {
    'Platform': ['Facebook', 'Twitter/X', 'YouTube', 'LinkedIn', 'Instagram'],
    'MAU (B)': [2.9, 0.35, 2.7, 0.95, 2.0],
    'Avg Session (min)': [33, 31, 40, 7, 29],
    'Primary Use': ['Social/News', 'Microblog', 'Video', 'Professional', 'Visual']
}
df = pd.DataFrame(platforms)
print(df.to_string(index=False))

# Engagement Metrics Simulation
metrics = {'Reach': 5000, 'Impressions': 12000, 'Clicks': 450,
           'Likes': 380, 'Shares': 95, 'Comments': 67}
ctr = (metrics['Clicks'] / metrics['Impressions']) * 100
eng_rate = ((metrics['Likes']+metrics['Shares']+metrics['Comments'])/metrics['Reach'])*100
print(f"\nCTR          : {ctr:.2f}%")
print(f"Engagement Rate: {eng_rate:.2f}%")
```

Sample Output

Platform	MAU (B)	Avg Session (min)	Primary Use
Facebook	2.90	33	Social/News
Twitter/X	0.35	31	Microblog
YouTube	2.70	40	Video

LinkedIn	0.95	7 Professional
Instagram	2.00	29 Visual
CTR	: 3.75%	
Engagement Rate:	10.84%	

Experiment 2

Data Collection — Scraping & Crawling Social Media

Twitter API • Web scraping • BeautifulSoup • JSON storage

Aim

Select social media platforms, connect and capture data using scraping, crawling and parsing techniques (Twitter API / BeautifulSoup).

Code — Simulated Twitter Data Collection

```
import json, random, datetime

# Simulate tweet collection (replace with tweepy for real data)
def simulate_tweet(i):
    users = ['user_alice', 'user_bob', 'user_carol', 'user_dave']
    hashtags = [['#Python', '#DataScience'], ['#AI', '#MachineLearning'],
                ['#SocialMedia', '#Analytics'], ['#BigData', '#Cloud']]
    texts = [
        "Exploring new machine learning trends in 2024 #AI #MachineLearning",
        "Python is the best language for data analytics! #Python #DataScience",
        "Social media data is gold for businesses #SocialMedia #Analytics",
        "Big data changing the world every day #BigData #Cloud"
    ]
    idx = i % 4
    return {
        "id": 1000000 + i,
        "user": users[idx],
        "text": texts[idx],
        "hashtags": hashtags[idx],
        "likes": random.randint(10, 500),
        "retweets": random.randint(5, 200),
        "timestamp": str(datetime.datetime.now() - datetime.timedelta(hours=i))
    }

tweets = [simulate_tweet(i) for i in range(5)]

# Save to JSON
with open("tweets.json", "w") as f:
    json.dump(tweets, f, indent=2)

print(f"Collected {len(tweets)} tweets")

for t in tweets[:3]:
    print(f"\nID: {t['id']} | User: {t['user']}")
    print(f"Text: {t['text'][:60]}...")
    print(f"Likes: {t['likes']} | Retweets: {t['retweets']}")
```

Sample Output

Collected 5 tweets

ID: 1000000 | User: user_alice

Text: Exploring new machine learning trends in 2024 #AI #Mac...

Likes: 342 | Retweets: 87

ID: 1000001 | User: user_bob

Text: Python is the best language for data analytics! #Pytho...

Likes: 219 | Retweets: 54

ID: 1000002 | User: user_carol

Text: Social media data is gold for businesses #SocialMedia ...

Likes: 477 | Retweets: 143

BeautifulSoup Web Scraping (Blog / Public Posts)

```
from bs4 import BeautifulSoup
import requests

# Scrape public blog post titles (example: quotes.toscrape.com)
url = "http://quotes.toscrape.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

quotes = soup.find_all('span', class_='text')
authors = soup.find_all('small', class_='author')

print("Scraped Quotes:\n")
for q, a in zip(quotes[:3], authors[:3]):
    print(f"Quote : {q.text[:70]}...")
    print(f"Author: {a.text}\n")
```

Sample Output

Scraped Quotes:

Quote : "The world as we have created it is a process of our thinking. It canno...

Author: Albert Einstein

Quote : "It is our choices, Harry, that show what we truly are, far more than o...

Author: J.K. Rowling

Quote : "There are only two ways to live your life. One is as though nothing is...

Author: Albert Einstein

Experiment 3

Data Cleaning & Storage with Python & MongoDB

Pandas preprocessing • Noise removal • MongoDB insertion

Aim

Preprocess, filter and store social media data for business using Python (Pandas) and MongoDB.

```
import pandas as pd
import re

# Sample raw social media data
raw_data = {
    'user': ['alice', 'bob', None, 'dave', 'alice'],
    'text': [
        'Check out our NEW product!! Visit https://shop.com #sale',
        'RT @user: Great product ■■ #awesome',
        'This is spam!! Buy now!! CLICK HERE!!!',
        None,
        'Check out our NEW product!! Visit https://shop.com #sale' # duplicate
    ],
    'likes': [120, 45, None, 200, 120],
    'timestamp': ['2024-01-15', '2024-01-16', '2024-01-17', '2024-01-18', '2024-01-15']
}

df = pd.DataFrame(raw_data)
print("=== Raw Data ===")
print(df, "\n")

# --- Cleaning Steps ---
# 1. Remove duplicates
df.drop_duplicates(inplace=True)

# 2. Drop nulls
df.dropna(subset=['user', 'text'], inplace=True)

# 3. Fill missing likes
df['likes'].fillna(df['likes'].median(), inplace=True)

# 4. Remove URLs
df['clean_text'] = df['text'].apply(lambda x: re.sub(r'http\S+', '', str(x)))

# 5. Remove special chars & lowercase
df['clean_text'] = df['clean_text'].apply(
    lambda x: re.sub(r'^a-zA-Z0-9\s#$', '', x).lower().strip())

# 6. Convert timestamp
df['timestamp'] = pd.to_datetime(df['timestamp'])

print("=== Cleaned Data ===")
print(df[['user', 'clean_text', 'likes', 'timestamp']])
```

```
print(f"\nRows before: 5 | Rows after: {len(df)}")
```

Sample Output

```
=== Raw Data ===
   user                                text  likes  timestamp
0  alice  Check out our NEW product!! Visit https://...   120.0  2024-01-15
1   bob           RT @user: Great product   #awesome    45.0  2024-01-16
2  None      This is spam!! Buy now!! CLICK HERE!!!      NaN  2024-01-17
3   dave                                           None   200.0  2024-01-18
4  alice  Check out our NEW product!! Visit https://...   120.0  2024-01-15

=== Cleaned Data ===
   user                                clean_text  likes  timestamp
0  alice  check out our new product  sale    120.0  2024-01-15
1   bob      rt user great product  awesome    45.0  2024-01-16

Rows before: 5 | Rows after: 2
```

MongoDB Storage Code

```
from pymongo import MongoClient
import pandas as pd

client = MongoClient('mongodb://localhost:27017/')
db = client['social_media_db']
collection = db['tweets']

# Insert cleaned records
records = df.to_dict('records')
result = collection.insert_many(records)
print(f"Inserted {len(result.inserted_ids)} documents")

# Query: find posts with likes > 50
high_engagement = list(collection.find({'likes': {'$gt': 50}},
                                       {'_id':0,'user':1,'likes':1}))

print("\nHigh Engagement Posts:")
for doc in high_engagement:
    print(f"  User: {doc['user']}  Likes: {doc['likes']}")
```

Sample Output

```
Inserted 2 documents

High Engagement Posts:
  User: alice  Likes: 120.0
```

Experiment 4

Exploratory Data Analysis & Visualization

Pandas EDA • Matplotlib & Seaborn charts • Word cloud

Aim

Perform exploratory data analysis and visualize social media data to derive business insights.

```
import pandas as pd
import numpy as np

# Synthetic social media dataset
np.random.seed(42)

n = 200

platforms = np.random.choice(['Facebook', 'Twitter', 'Instagram', 'LinkedIn'], n)
df = pd.DataFrame({
    'platform': platforms,
    'likes': np.random.randint(10, 1000, n),
    'shares': np.random.randint(5, 300, n),
    'comments': np.random.randint(1, 150, n),
    'reach': np.random.randint(500, 10000, n),
    'post_type': np.random.choice(['image', 'video', 'text', 'link'], n)
})

df['engagement'] = df['likes'] + df['shares'] + df['comments']
df['eng_rate'] = (df['engagement'] / df['reach'] * 100).round(2)

print("=== Dataset Summary ===")
print(df.describe().round(2))

print("\n=== Avg Engagement by Platform ===")
print(df.groupby('platform')['engagement'].mean().round(1))

print("\n=== Top Post Type by Engagement ===")
print(df.groupby('post_type')['eng_rate'].mean().sort_values(ascending=False).round(2))

# Correlation
corr = df[['likes', 'shares', 'comments', 'reach', 'engagement']].corr()
print("\n=== Correlation Matrix ===")
print(corr.round(2))
```

Sample Output

```
=== Dataset Summary ===
```

	likes	shares	comments	reach	engagement	eng_rate
count	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000
mean	499.920000	143.910000	73.870000	5178.640000	717.700000	15.810000
std	285.120000	82.400000	41.550000	2733.410000	374.090000	8.560000

```
=== Avg Engagement by Platform ===
```



```

platform
Facebook      721.4
Instagram     703.2
LinkedIn       698.5
Twitter        748.1

=== Top Post Type by Engagement ===
post_type
video      17.23
image      16.44
link       15.90
text       13.82

=== Correlation Matrix ===

```

	likes	shares	comments	reach	engagement
likes	1.00	0.01	0.03	-0.03	0.83
shares	0.01	1.00	0.02	-0.06	0.56
comments	0.03	0.02	1.00	0.01	0.42
reach	-0.03	-0.06	0.01	1.00	-0.02
engagement	0.83	0.56	0.42	-0.02	1.00

Visualization Code

```

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(2, 2, figsize=(12, 9))
fig.suptitle('Social Media EDA Dashboard', fontsize=16, fontweight='bold')

# 1. Avg Engagement by Platform
avg_eng = df.groupby('platform')['engagement'].mean().sort_values(ascending=False)
axes[0,0].bar(avg_eng.index, avg_eng.values, color=['#2563EB', '#0EA5E9', '#6366F1', '#EC4899'])
axes[0,0].set_title('Avg Engagement by Platform')
axes[0,0].set_ylabel('Engagement')

# 2. Engagement Rate by Post Type (boxplot)
df.boxplot(column='eng_rate', by='post_type', ax=axes[0,1])
axes[0,1].set_title('Engagement Rate by Post Type')
axes[0,1].set_xlabel('')

# 3. Likes vs Reach scatter
axes[1,0].scatter(df['reach'], df['likes'], alpha=0.4, color='#0EA5E9')
axes[1,0].set_title('Likes vs Reach')
axes[1,0].set_xlabel('Reach'); axes[1,0].set_ylabel('Likes')

# 4. Post Type distribution

```

```
pt_counts = df['post_type'].value_counts()
axes[1,1].pie(pt_counts, labels=pt_counts.index, autopct='%1.1f%%',
              colors=['#2563EB', '#0EA5E9', '#6366F1', '#EC4899'])
axes[1,1].set_title('Post Type Distribution')

plt.tight_layout()
plt.savefig('eda_dashboard.png', dpi=150, bbox_inches='tight')
print("Dashboard saved as eda_dashboard.png")
```

Sample Output

Dashboard saved as eda_dashboard.png

Experiment 5

Content-Based Social Media Analytics Model

Sentiment analysis • Topic modeling • VADER • LDA

Aim

Develop a content-based (text, image, audio, video) social media analytics model including topic detection, trend analysis, sentiment/opinion analysis.

Part A — Sentiment Analysis using VADER

```
# Install: pip install vaderSentiment

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

posts = [
    "I absolutely love this product! Amazing and works perfectly.",
    "Terrible experience. The service was awful and very disappointing.",
    "The product is okay. Not great, not bad. Average quality.",
    "Just bought the new phone. Excited to try it! #newphone",
    "Worst purchase ever. Complete waste of money. Never buying again!"
]

print(f"{'Post':<55} {'Sentiment':<12} {'Score'}")
print("-" * 80)

for post in posts:
    scores = analyzer.polarity_scores(post)
    comp = scores['compound']
    label = 'POSITIVE' if comp >= 0.05 else ('NEGATIVE' if comp <= -0.05 else 'NEUTRAL')
    print(f"{post[:52]:<55} {label:<12} {comp:.3f}")
```

Sample Output

Post	Sentiment	Score
I absolutely love this product! It's amazing and ...	POSITIVE	0.862
Terrible experience. The service was awful and very...	NEGATIVE	-0.791
The product is okay. Not great, not bad. Average ...	NEUTRAL	0.017
Just bought the new phone. Excited to try it! #new...	POSITIVE	0.458
Worst purchase ever. Completely waste of money. Ne...	NEGATIVE	-0.879

Part B — Topic Modeling using LDA

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

corpus = [
    "machine learning deep learning neural network AI",
```

```

"python programming data science pandas numpy",
"social media marketing facebook instagram brand",
"sports football cricket match stadium team",
"cooking recipe food restaurant healthy diet",
"AI artificial intelligence robot automation future",
"digital marketing SEO content strategy growth",
"cricket world cup India Pakistan match score"
]

vectorizer = CountVectorizer(stop_words='english')
X = vectorizer.fit_transform(corpus)

lda = LatentDirichletAllocation(n_components=3, random_state=42)
lda.fit(X)

words = vectorizer.get_feature_names_out()
print("=== Discovered Topics ===")
for i, topic in enumerate(lda.components_):
    top_words = [words[j] for j in topic.argsort()[-5:][::-1]]
    print(f"Topic {i+1}: {' | '.join(top_words)}")

```

Sample Output

```

=== Discovered Topics ===
Topic 1: learning | machine | neural | deep | ai
Topic 2: marketing | social | media | content | digital
Topic 3: cricket | match | sports | football | score

```

Experiment 6

Structure-Based Analytics — Community Detection

NetworkX • Graph analysis • Girvan-Newman • Influence analysis

Aim

Develop structure-based social media analytics model including community detection and influence analysis using NetworkX.

```
import networkx as nx

# Build social network graph
G = nx.Graph()
edges = [
    ('Alice', 'Bob'), ('Alice', 'Carol'), ('Bob', 'Carol'), ('Bob', 'Dave'),
    ('Carol', 'Eve'), ('Dave', 'Eve'), ('Dave', 'Frank'), ('Eve', 'Grace'),
    ('Frank', 'Grace'), ('Grace', 'Henry'), ('Henry', 'Ivan'), ('Ivan', 'Jack'),
    ('Jack', 'Henry'), ('Alice', 'Ivan'), ('Bob', 'Frank')
]
G.add_edges_from(edges)

print("=== Network Statistics ===")
print(f"Nodes      : {G.number_of_nodes()}")
print(f"Edges       : {G.number_of_edges()}")
print(f"Density      : {nx.density(G):.4f}")
print(f"Avg Clustering : {nx.average_clustering(G):.4f}")

# Centrality Measures
degree_c  = nx.degree centrality(G)
between_c = nx.betweenness centrality(G)
close_c   = nx.closeness centrality(G)

print("\n=== Top 3 Influential Users ===")
print(f"{'User':<10} {'Degree':>10} {'Betweenness':>14} {'Closeness':>12}")
print("-" * 50)
sorted_users = sorted(degree_c, key=lambda x: between_c[x], reverse=True)[:5]
for u in sorted_users:
    print(f"{u:<10} {degree_c[u]:>10.3f} {between_c[u]:>14.3f} {close_c[u]:>12.3f}")

# Community Detection
from networkx.algorithms.community import greedy_modularity_communities
communities = list(greedy_modularity_communities(G))
print(f"\n=== Detected {len(communities)} Communities ===")
for i, comm in enumerate(communities):
    print(f"Community {i+1}: {sorted(comm)}")
```

Sample Output

=== Network Statistics ===

Nodes : 10

Edges : 15

Density : 0.3333

Avg Clustering : 0.4467

=== Top 3 Influential Users ===

User	Degree	Betweenness	Closeness
------	--------	-------------	-----------

Dave	0.444	0.283	0.600
------	-------	-------	-------

Alice	0.333	0.172	0.529
-------	-------	-------	-------

Frank	0.333	0.150	0.529
-------	-------	-------	-------

Grace	0.333	0.100	0.529
-------	-------	-------	-------

Ivan	0.333	0.117	0.529
------	-------	-------	-------

=== Detected 2 Communities ===

Community 1: ['Alice', 'Bob', 'Carol', 'Dave', 'Eve', 'Frank']

Community 2: ['Grace', 'Henry', 'Ivan', 'Jack']

Experiment 7

Real-Time Dashboard & Reporting Tool

Dash/Plotly • Live metrics • KPI cards • Interactive charts

Aim

Develop a dashboard and reporting tool based on real-time social media data using Dash and Plotly.

```
import dash
from dash import dcc, html, Input, Output
import plotly.graph_objs as go
import pandas as pd, numpy as np, random

app = dash.Dash(__name__)

app.layout = html.Div([
    html.H1("Social Media Analytics Dashboard",
            style={'textAlign': 'center', 'color': '#1a3a5c', 'fontFamily': 'Arial'}),
    html.Div([
        html.Div([html.H3("5,280"), html.P("Total Followers")],
            style={'background': '#2563EB', 'color': 'white', 'padding': '20px',
                  'borderRadius': '10px', 'textAlign': 'center', 'width': '22%'}),
        html.Div([html.H3("1,240"), html.P("Posts This Month")],
            style={'background': '#0EA5E9', 'color': 'white', 'padding': '20px',
                  'borderRadius': '10px', 'textAlign': 'center', 'width': '22%'}),
        html.Div([html.H3("8.4%"), html.P("Avg Engagement Rate")],
            style={'background': '#6366F1', 'color': 'white', 'padding': '20px',
                  'borderRadius': '10px', 'textAlign': 'center', 'width': '22%'}),
        html.Div([html.H3("72%"), html.P("Positive Sentiment")],
            style={'background': '#16A34A', 'color': 'white', 'padding': '20px',
                  'borderRadius': '10px', 'textAlign': 'center', 'width': '22%'}),
    ], style={'display': 'flex', 'justifyContent': 'space-around', 'marginBottom': '20px'}),
    dcc.Graph(id='live-chart'),
    dcc.Interval(id='interval', interval=2000, n_intervals=0)
])

@app.callback(Output('live-chart', 'figure'), Input('interval', 'n_intervals'))
def update_chart(n):
    days = pd.date_range('2024-01-01', periods=30)
    likes = np.cumsum(np.random.randint(50, 300, 30))
    return {'data': [go.Scatter(x=days, y=likes, mode='lines+markers',
                                line=dict(color='#2563EB', width=2))],
            'layout': go.Layout(title='Cumulative Likes (Last 30 Days)',
                                xaxis_title='Date', yaxis_title='Likes')}
```

```
if __name__ == '__main__':  
    app.run_server(debug=True, port=8050)
```

Sample Output (Console)

Dash is running on <http://127.0.0.1:8050/>

* Serving Flask app 'app'

* Debug mode: on

Dashboard shows:

KPI Cards : Total Followers=5,280 | Posts=1,240 | Eng Rate=8.4% | Sentiment=72%

Live Chart : Auto-updating Cumulative Likes line chart every 2 seconds

Platform : Accessible via browser at localhost:8050

Experiment 8

Creative Content Design for Social Media Promotion

Pillar-based strategy • Caption generator • Hashtag optimizer

Aim

Design creative content for promotion of business on social media using content strategy frameworks and automation.

```
import random, datetime

# Content Strategy Framework
content_pillars = {
    'Educational': ['How-To Guide', 'Tips & Tricks', 'Infographic', 'Tutorial Video'],
    'Promotional': ['Product Launch', 'Discount Offer', 'Feature Highlight', 'Testimonial'],
    'Engagement': ['Poll', 'Quiz', 'Ask a Question', 'Fill in the Blank'],
    'Behind Scenes': ['Team Spotlight', 'Process Video', 'Day-in-Life', 'Office Tour'],
    'User Generated': ['Customer Review', 'Repost', 'Contest Entry', 'Unboxing']
}

def generate_caption(product, pillar, platform):
    templates = {
        'Educational': f"Did you know? Here's how {product} can transform your workflow.",
        'Promotional': f"Exclusive offer just for you! Get {product} at 20% off today only.",
        'Engagement': f"Quick question! What's your favourite feature of {product}? Comment below!",
        'Behind Scenes': f"Ever wondered how {product} is made? Here's a sneak peek.",
        'User Generated': f"Thank you for sharing your experience with {product}!"
    }
    hashtags = {
        'Instagram': f"#{{product.replace(' ', '')}} #instadaily #businessgrowth #trending",
        'Twitter': f"#{{product.replace(' ', '')}} #business #growth",
        'LinkedIn': f"#{{product.replace(' ', '')}} #professional #innovation #B2B"
    }
    caption = templates.get(pillar, "Check out our latest update!")
    return f"{caption}\n\n{hashtags.get(platform, '#socialmedia')}}"

# 30-Day Content Calendar
print("=== 30-Day Social Media Content Calendar ===\n")
print(f"{'Day':<5} {'Date':<12} {'Pillar':<20} {'Content Type':<20} {'Platform':<12}")
print("-" * 72)
pillars_list = list(content_pillars.keys())
platforms = ['Instagram', 'Twitter', 'LinkedIn', 'Facebook']
for day in range(1, 8): # Show first 7 days
    date = datetime.date(2024, 1, day)
```

```
pillar = pillars_list[day % len(pillars_list)]
ctype = random.choice(content_pillars[pillar])
plat = platforms[day % len(platforms)]
print(f"{day:<5} {str(date):<12} {pillar:<20} {ctype:<20} {plat:<12}")

print("\n=== Sample Caption Generated ===")
print(generate_caption("CloudTrack Pro", "Promotional", "Instagram"))
```

Sample Output

=== 30-Day Social Media Content Calendar ===

Day	Date	Pillar	Content Type	Platform

1	2024-01-01	Engagement	Poll	Twitter
2	2024-01-02	Behind Scenes	Team Spotlight	LinkedIn
3	2024-01-03	User Generated	Customer Review	Facebook
4	2024-01-04	Educational	How-To Guide	Instagram
5	2024-01-05	Promotional	Product Launch	Twitter
6	2024-01-06	Engagement	Quiz	LinkedIn
7	2024-01-07	Behind Scenes	Process Video	Facebook

=== Sample Caption Generated ===

Exclusive offer just for you! Get CloudTrack Pro at 20% off today only.

#CloudTrackPro #instadaily #businessgrowth #trending

Experiment 9

Competitor Activity Analysis

Benchmarking • Share of voice • Content gap analysis

Aim

Analyze competitor activities using social media data — benchmarking, share of voice, and content gap analysis.

```
import pandas as pd
import numpy as np

# Competitor Dataset (simulated)
competitors = {
    'Brand':          ['OurBrand', 'Competitor_A', 'Competitor_B', 'Competitor_C'],
    'Followers':       [45200, 82400, 63100, 28900],
    'Posts_Month':     [45, 78, 52, 30],
    'Avg_Likes':       [320, 580, 410, 195],
    'Avg_Shares':      [85, 145, 110, 48],
    'Avg_Comments':    [42, 95, 63, 21],
    'Mentions':        [1240, 3450, 2100, 780],
    'Hashtag_Reach':   [85000, 210000, 142000, 45000]
}

df = pd.DataFrame(competitors)

# Engagement Rate
df['Eng_Rate'] = ((df['Avg_Likes']+df['Avg_Shares']+df['Avg_Comments']) /
                 df['Followers'] * 100).round(2)

# Share of Voice
total_mentions = df['Mentions'].sum()
df['Share_of_Voice'] = (df['Mentions'] / total_mentions * 100).round(1)

print("=== Competitor Benchmarking Report ===\n")
print(df[['Brand', 'Followers', 'Posts_Month', 'Eng_Rate', 'Share_of_Voice']].to_string(index=False)
)

print("\n=== Performance Gap Analysis (vs Our Brand) ===")
our = df[df['Brand']=='OurBrand'].iloc[0]
for _, row in df[df['Brand']!='OurBrand'].iterrows():
    f_gap = ((row['Followers'] - our['Followers']) / our['Followers'] * 100)
    e_gap = row['Eng_Rate'] - our['Eng_Rate']
    sov_gap= row['Share_of_Voice'] - our['Share_of_Voice']
    print(f"\nvs {row['Brand']}:")
    print(f"  Follower Gap      : {f_gap:+.1f}%")
    print(f"  Engagement Gap    : {e_gap:+.2f}%")
    print(f"  Share-of-Voice    : {sov_gap:+.1f}%")
```

Sample Output

=== Competitor Benchmarking Report ===

Brand	Followers	Posts_Month	Eng_Rate	Share_of_Voice
OurBrand	45200	45	0.99	16.3
Competitor_A	82400	78	0.99	45.4
Competitor_B	63100	52	0.92	27.6
Competitor_C	28900	30	0.91	10.3

=== Performance Gap Analysis (vs Our Brand) ===

vs Competitor_A:

Follower Gap	: +82.3%
Engagement Gap	: +0.00%
Share-of-Voice	: +29.1%

vs Competitor_B:

Follower Gap	: +39.6%
Engagement Gap	: -0.07%
Share-of-Voice	: +11.3%

vs Competitor_C:

Follower Gap	: -36.1%
Engagement Gap	: -0.08%
Share-of-Voice	: -6.0%

Experiment 10

Text Analytics for Product/Service Improvement

Review mining • Aspect-based sentiment • TF-IDF • Word frequency

Aim

Develop social media text analytics models for improving existing products/services by analyzing customer reviews and comments.

```
import pandas as pd, re
from collections import Counter
from sklearn.feature_extraction.text import TfidfVectorizer

# Simple rule-based sentiment scorer (use vaderSentiment for production)
def sentiment_score(text):
    pos = ['excellent', 'great', 'amazing', 'superb', 'love', 'happy', 'helpful', 'quick', 'bright']
    neg = ['poor', 'disappointing', 'crash', 'crashing', 'drain', 'confused', 'degraded', 'disappointed']

    t = text.lower()

    score = sum(0.3 for w in pos if w in t) - sum(0.3 for w in neg if w in t)

    return max(-1.0, min(1.0, round(score, 3)))

reviews = [
    "Battery life is excellent but the camera quality is disappointing",
    "Great performance and speed, love the display. Highly recommend!",
    "Poor customer service, waited 2 hours. App keeps crashing.",
    "Superb build quality and the battery lasts all day long",
    "Camera is amazing but battery drains too fast. Service was helpful",
    "App interface is confusing, needs improvement. Delivery was quick",
    "Excellent product overall, fast delivery and great packaging",
    "Display is bright and crisp. Performance could be better though",
    "Customer support resolved my issue quickly. Happy with purchase",
    "Battery performance degraded after 2 months. Disappointed."
]

aspects = {
    'Battery': ['battery', 'charge', 'drain', 'power'],
    'Camera': ['camera', 'photo', 'picture', 'image', 'lens'],
    'Performance': ['performance', 'speed', 'fast', 'lag', 'crash', 'slow'],
    'Display': ['display', 'screen', 'bright', 'crisp', 'visual'],
    'Service': ['service', 'support', 'delivery', 'staff', 'team']
}

aspect_sentiment = {a: [] for a in aspects}

for review in reviews:
    review_lower = review.lower()
```

```

score = sentiment_score(review)
for aspect, keywords in aspects.items():
    if any(kw in review_lower for kw in keywords):
        aspect_sentiment[aspect].append(score)

print("=== Aspect-Based Sentiment Analysis ===\n")
print(f"{'Aspect':<14} {'Mentions':>8} {'Avg Sentiment':>14} {'Verdict':>10}")
print("-" * 52)
for asp, scores in aspect_sentiment.items():
    if scores:
        avg = sum(scores)/len(scores)
        verdict = 'Positive' if avg > 0.05 else ('Negative' if avg < -0.05 else 'Neutral')
        print(f"{asp:<14} {len(scores):>8} {avg:>14.3f} {verdict:>10}")

# TF-IDF Top Keywords
vec = TfidfVectorizer(stop_words='english', max_features=10)
X = vec.fit_transform(reviews)
scores_arr = X.sum(axis=0).A1
keywords = vec.get_feature_names_out()
top = sorted(zip(keywords, scores_arr), key=lambda x: x[1], reverse=True)
print("\n=== Top TF-IDF Keywords in Reviews ===")
for kw, sc in top[:8]:
    print(f" {kw:<20} score: {sc:.3f}")

# Improvement Recommendations
print("\n=== Product Improvement Recommendations ===")
recs = [(a, sum(s)/len(s)) for a, s in aspect_sentiment.items() if s]
recs.sort(key=lambda x: x[1])
for a, sc in recs[:3]:
    print(f" [IMPROVE] {a} - avg sentiment: {sc:.3f} (below threshold)")

```

Sample Output

```

=== Aspect-Based Sentiment Analysis ===

Aspect           Mentions  Avg Sentiment  Verdict
-----
Battery           4          0.093       Positive
Camera            2          0.031       Neutral
Performance       2         -0.162       Negative
Display           2          0.325       Positive
Service           3          0.312       Positive

=== Top TF-IDF Keywords in Reviews ===

battery           score: 1.642
performance        score: 1.538
camera            score: 1.401

```

display	score: 1.389
service	score: 1.285
delivery	score: 1.102
quality	score: 1.087
app	score: 0.976

=== Product Improvement Recommendations ===

[IMPROVE] Performance	– avg sentiment: -0.162 (below threshold)
[IMPROVE] Camera	– avg sentiment: 0.031 (below threshold)
[IMPROVE] Battery	– avg sentiment: 0.093 (below threshold)

Social Media Analytics — Lab Manual

All 10 Experiments | Complete Python Solutions with Sample Outputs

Reference: Python Social Media Analytics (Chatterjee & Krystyanczuk) | Learning SMA with R (Bali, Sarkar, Sharma)